

Data Mining Assignments 1 and 2

Detailed Presentation Script

Autenia Murray
CSCI 7434: Data Mining
Spring 2026

This script is written in a conversational style so it can be read directly during the Zoom presentation. It includes formulas, actual example calculations, variable meanings, and short code-demo snippets.

Data Mining Assignments 1 and 2, Detailed Presentation Script

Name: Autenia Murray

Course: CSCI 7434, Data Mining

Term: Spring 2026

Purpose: This is a read-aloud presentation script for the live Zoom explanation. It is written conversationally so it can be read naturally while still showing formulas, actual example calculations, variable meanings, and what each process is doing.

How to use this script

Use the code-demo snippets before running each program. Then use the section scripts when explaining the report tables. The formula maps are written so you can point to a table, say the formula, explain the variables, and show one actual calculation without sounding like you are reading a textbook.

Opening Script for Both Assignments

Good evening, Dr. Hashemi. I am going to present Assignment 1 and Assignment 2. For both assignments, my goal was to build a pipeline that runs from the beginning to the end, saves intermediate outputs, and makes the calculations easy to inspect. I used the reports and Excel outputs as support files, so if you ask about a specific calculation, I can open the matching table and show exactly where it came from.

For Assignment 1, I am using the notebook-style version because that is the version I used for the presentation report. For Assignment 2, I am using the VS Code Python pipeline because the assignment asks me to run the program, choose ID3 or Naive Bayes, and then let the program run through the selected method.

Code Demo Snippet Before Running Assignment 1

Before I run Assignment 1, I want to explain what the program is about. This pipeline starts by asking for the user-selected thresholds. After that, it runs through outlier detection, correlation analysis, entropy-based discretization, Apriori frequent itemset mining, association rule generation, and prediction. The reason I collected the inputs at the beginning is so the program does not stop in the middle of the run. The report and output tables show the calculations used at each major step.

If the professor asks why the notebook is being used:

I used the notebook version for Assignment 1 because it shows the build process section by section. The logic is still a Python pipeline, but the notebook format makes it easier to show the stages during the presentation.

Assignment 1 Presentation Script

1. Program Initialization Inputs

I started Assignment 1 by collecting the user-selected values. The Pearson correlation threshold was 0.70, the minimum support count was 15, the confidence threshold was 0.70, and the imbalance ratio threshold was fixed at 0.40. These values control different parts of the pipeline. The correlation threshold decides when two attributes are considered redundant. The support count decides which itemsets are frequent enough for Apriori. The confidence threshold decides which rules are strong enough to keep. The imbalance ratio decides whether I use standard confidence or Kulc confidence.

Assignment 1 Initialization Formula Map

Process	Formula or rule	Example from the report	What each variable means	What I would say out loud
Correlation threshold	If $\text{abs}(r) > \text{threshold}$, mark as highly correlated.	threshold = 0.70	r = Pearson correlation coefficient. threshold = user-selected cutoff.	If the absolute correlation is above 0.70, I treat the second attribute as redundant because it is carrying very similar information.
Minimum support	Keep itemset if $\text{support_count} \geq \text{min_support}$.	min_support = 15	support_count = number of Econ-TRAIN transactions containing the itemset.	An itemset has to appear at least 15 times in the training transactions to be considered frequent.
Minimum confidence	Keep rule if $\text{confidence} \geq \text{min_confidence}$.	min_confidence = 0.70, or 70%.	confidence = how often the consequent appears when the antecedent appears.	A rule must be correct at least 70% of the time on the training transactions to survive.
Imbalance ratio threshold	If $\text{IR} > 0.40$, treat dataset as imbalanced.	IR threshold = 0.40	IR = class imbalance ratio.	This decides whether regular confidence is safe or whether Kulc is needed because of imbalance.

2.1 Outlier Detection

The first preprocessing step is outlier detection. The program checks each independent attribute separately. For each attribute, it calculates the average and population standard deviation. Then it builds a lower and upper boundary. If a value is outside those boundaries, it is marked as an outlier. The program removes the whole row if any independent attribute in that row is an outlier, because one extreme value can distort later correlation, discretization, and rule mining calculations.

Assignment 1, 2.1 Formula Map, Outlier Detection

Process	Formula	Example from the report or code	What each variable means	What I would say out loud
Upper outlier boundary	$\text{upper_bound} = \text{AVG} + 2 * \text{STD}$	The code calculates $\text{attr_mean} + (2 * \text{attr_std})$ for each continuous attribute.	AVG = average value for the attribute. STD = population standard deviation.	This gives the high cutoff. Anything above this boundary is considered unusually high.
Lower outlier boundary	$\text{lower_bound} = \text{AVG} - 2 * \text{STD}$	The code calculates $\text{attr_mean} - (2 * \text{attr_std})$ for each continuous attribute.	AVG = average value for the attribute. STD = population standard deviation.	This gives the low cutoff. Anything below this boundary is considered unusually low.
Remove outlier rows	cleaned_data = original_data minus rows with outlier values	The notebook removes rows from clean_indices when an anomaly is found.	clean_indices = rows still kept. anomalies.index = rows where an outlier was found.	I remove the full record instead of just changing the value because the later mining steps depend on complete records.

2.2 Correlation Analysis

After removing outliers, the program checks the remaining continuous attributes for redundancy. I used Pearson correlation because the assignment dealt with numeric attributes in this part. The user-selected threshold was 0.70. If the absolute value of a correlation coefficient is greater than 0.70, the program treats that pair as highly correlated and removes the later attribute from the reduced dataset.

Assignment 1, 2.2 Formula Map, Correlation

Process	Formula or rule	Example from the report	What each variable means	What I would say out loud
Pearson redundancy check	If $\text{abs}(r) > 0.70$, mark as redundant.	User threshold = 0.70.	r = Pearson correlation coefficient. $\text{abs}(r)$ means positive and negative strong relationships both count.	I am not only looking for positive correlation. If two attributes move strongly together in either direction, one may be redundant.

Reduced dataset	reduced_data = cleaned_data - redundant_attributes	The code drops any attribute identified as redundant.	cleaned_data = data after outlier removal. redundant_attributes = attributes above the threshold.	This helps prevent the later mining stage from counting the same signal twice.
-----------------	---	--	---	---

2.3 Entropy-Based Discretization

The next step converts continuous values into discrete values. Apriori needs item-style values, so the numeric columns must be converted into bins. I used the Diff_Union approach. For each attribute, the program sorts the unique values, calculates differences between adjacent values, finds the largest differences, builds candidate split thresholds, and then selects the threshold that gives the highest information gain.

Assignment 1, 2.3 Formula Map, Discretization

Process	Formula	Example from the report or code	What each variable means	What I would say out loud
Adjacent difference	difference = high_value - low_value	diff_val = round(sorted_vals[idx] - sorted_vals[idx + 1], 1)	high_value and low_value are adjacent sorted values.	I am looking for larger gaps because a large gap may be a useful place to split the data.
Candidate thresholds	Diff_Union = lower values tied to the largest differences.	The code keeps the lower value from rows whose difference matches one of the top three difference values.	Diff_Union = threshold candidate set.	This limits the threshold search to candidate values that are likely to separate different groups.
Entropy or MC	$MC(D) = -\sum(p_i \log_2(p_i))$	The notebook uses calc_entropy(data_subset).	D = current dataset. p_i = proportion of each class label.	Entropy measures how mixed the class labels are.
Weighted MC	$WMC = (n1/n)MC(S1) + (n2/n)MC(S2)$	The code calculates weighted entropy for the lower and upper partitions.	n1 and n2 are partition sizes. n is total records.	This tells me how mixed the data would be after splitting at a threshold.
Gain	Gain = MC(D) - WMC	The code keeps the threshold with the largest gain.	Gain = how much cleaner the split makes the class labels.	The chosen threshold is the one that reduces class mixture the most.
Binary discretization	value = 1 if original value < threshold, otherwise 2	np.where(attribute < threshold, 1, 2)	threshold = selected split point.	After this, the numeric attribute becomes a discrete item value for Apriori.

3.1 Discretization Mapping and Table A

After discretization, the program maps every attribute-value pair to an I-notation item. For example, one item may represent a specific attribute with discrete value 1, and another item may represent the same attribute with discrete value 2. The dependent variable treat is also mapped to I-items, because the association rules need to predict treat as the conclusion.

Assignment 1, 3.1 Formula Map, I-Notation

Process	Formula or rule	Example from the report or code	What each variable means	What I would say out loud
I-notation map	(attribute, discrete_value) -> I-number	(attribute, discrete_val) maps to I1, I2, I3, and so on.	attribute = column name. discrete_value = 1 or 2 for independent attributes.	I-notation turns each table value into an item Apriori can count.
Treat mapping	(treat, 0) and (treat, 1) also map to I-items.	The code maps both treat class values.	treat = dependent variable.	This allows the rule conclusion to predict the class value.
Transaction conversion	Each row becomes a list of I-items.	mapped_data stores I-items instead of raw values.	One transaction = one record represented as item IDs.	Once the rows are converted, Apriori can count itemsets.

3.2 Train-Test Split and Balance Check

The program creates a stratified test set by sampling 10% from each treat class. I used a fixed random state so the split is repeatable. Then I calculated the imbalance ratio. In the report, Econ-TRAIN has 155 records with treat = 1 and 123 records with treat = 0. That gives an imbalance ratio of 0.1151, which is less than 0.40. So the dataset is treated as balanced and standard confidence is used instead of Kulc.

Assignment 1, 3.2 Formula Map, Split and Balance

Process	Formula	Example from the report	What each variable means	What I would say out loud
---------	---------	-------------------------	--------------------------	---------------------------

Stratified test split	Econ-TEST = 10% from each treat class	sample(frac=0.1, random_state=42)	frac = 0.1 means 10%. random_state makes the selection repeatable.	I used a repeatable split so the same confusion matrix appears during the live demo.
Imbalance ratio	$IR = \text{abs}(\text{count1} - \text{count0}) / \text{total_train}$	$IR = \text{abs}(155 - 123) / 278 = 32 / 278 = 0.1151$	count1 = treat 1 count. count0 = treat 0 count. total_train = 278.	Since 0.1151 is below 0.40, I treat this as balanced.
Metric choice	If $IR \leq 0.40$, use standard confidence. If $IR > 0.40$, use Kulc.	$0.1151 \leq 0.40$, so standard confidence is used.	IR threshold = 0.40.	The balance check decides which rule scoring formula is appropriate.

3.3 Apriori Frequent Itemsets

Apriori finds combinations of I-items that appear often enough in Econ-TRAIN. I used a minimum support count of 15. So any itemset must appear in at least 15 training transactions to survive. I also removed itemsets that contain two values from the same attribute, because one record cannot logically have both values for the same column.

Assignment 1, 3.3 Formula Map, Apriori

Process	Formula or rule	Example from the report	What each variable means	What I would say out loud
Frequent itemset rule	$\text{support_count} \geq \text{minimum_support_count}$	$\text{support_count} \geq 15$	support_count = number of training transactions containing the itemset.	This removes combinations that are too rare to support reliable rules.
Invalid itemset rule	Remove itemsets with more than one value from the same attribute.	Example: debt = 1 and debt = 2 cannot both be true for the same row.	duplicate attribute = same column appears twice in one itemset.	This keeps the itemsets logically valid.
Candidate expansion	Larger candidate itemsets are built from smaller frequent itemsets.	Apriori grows level by level.	A larger itemset can only survive if its smaller parts are frequent.	This is how Apriori avoids searching every possible combination blindly.

3.4 Association Rule Generation

After the frequent itemsets are found, the program turns them into rules. A rule has an antecedent on the left side and a consequent on the right side. Since this is predictive modeling, I only keep rules where the conclusion predicts treat. Because the dataset was balanced, I used standard confidence. If the data had been imbalanced, I would have used Kulc confidence.

Assignment 1, 3.4 Formula Map, Association Rules

Process	Formula	Example from the report	What each variable means	What I would say out loud
Standard confidence	$\text{confidence}(A \rightarrow B) = \text{count}(A \text{ and } B) / \text{count}(A)$	Used because $IR = 0.1151$, which is balanced.	A = antecedent. B = consequent.	Confidence asks, when the left side appears, how often does the right side also appear?
Kulc confidence	$\text{Kulc} = 0.5 * [\text{count}(A \text{ and } B) / \text{count}(A) + \text{count}(A \text{ and } B) / \text{count}(B)]$	Not used here because the data was balanced.	Kulc averages both directional confidence values.	Kulc is useful when class imbalance could make regular confidence misleading.
Confidence filter	keep rule if confidence ≥ 0.70	minimum confidence = 70%.	confidence threshold = user-selected cutoff.	Only rules with at least 70% confidence survive.
Format-1 rule filter	consequent must be exactly one predicate and must be treat	Consequent predicate count = 1, consequent attribute = treat.	consequent = right side of rule.	This makes the rule usable for prediction because it assigns one class value.

4. Prediction and Results

The final Assignment 1 step applies the Format-1 rules to Econ-TEST. The official result uses no default rule because the original assignment did not specify a fallback. In the report, the model predicts treat = 1 for all 31 test records. That gives 17 true positives, 0 false negatives, 14 false positives, and 0 true negatives. The accuracy is 54.84%. The recall is 100% because all actual treat = 1 records were found. The precision is 54.84% because 14 of the positive predictions were actually treat = 0.

Assignment 1, 4 Formula Map, Results

Process	Formula	Actual calculation from the report	What each variable means	What I would say out loud
True positive	X = actual 1 and predicted 1	X = 17	X = true positives.	The model correctly predicted 17 actual treat = 1 records.
False negative	Y = actual 1 and predicted 0	Y = 0	Y = missed positives.	The model did not miss any treat = 1 records.
False positive	Z = actual 0 and predicted 1	Z = 14	Z = records incorrectly predicted as positive.	The model incorrectly labeled all 14 actual treat

True negative	W = actual 0 and predicted 0	W = 0	W = correctly predicted negatives.	= 0 records as treat = 1. The model did not correctly identify any treat = 0 records.
Accuracy	$\text{accuracy} = (X + W) / (X + Y + Z + W)$	$(17 + 0) / (17 + 0 + 14 + 0) = 17 / 31 = 54.84\%$	X + W = correct predictions. denominator = all predictions.	Accuracy tells me how often the model was right overall.
Recall	$\text{recall} = X / (X + Y)$	$17 / (17 + 0) = 17 / 17 = 100\%$	X = found positives. Y = missed positives.	Recall is perfect here because every actual treat = 1 record was found.
Precision	$\text{precision} = X / (X + Z)$	$17 / (17 + 14) = 17 / 31 = 54.84\%$	Z = false positives.	Precision is lower because the model predicted treat = 1 too often.

Assignment 1 Closing Explanation

So my Assignment 1 result is valid, but not perfect. The model mostly learned rules that predict treat = 1. That explains the high recall and lower precision. The important point is that the pipeline followed the required steps: outlier removal, correlation analysis, discretization, Apriori, rule generation, rule filtering, and prediction.

Code Demo Snippet Before Running Assignment 2

Before I run Assignment 2, I want to explain the structure. This is a VS Code Python pipeline. At the beginning, the program asks whether the company wants to use ID3 or Naive Bayes. If ID3 is selected, it asks for the pre-pruning threshold T1 and the post-pruning parameter g. Once those values are entered, the program runs the selected method from start to finish, saves Excel files for the major calculation steps, and creates report files. I used my own implementation for ID3 and Naive Bayes calculations. The external libraries are only used for file handling and report output, not for classifier logic.

If I choose ID3, I will enter T1 = 0.80 and g = 0.010 because those are the values reflected in the submitted output files. If I choose Bayes, there are no pruning thresholds, so the program runs after the classifier and live-output choices are entered.

Assignment 2 Presentation Script

2. Dataset Overview and Method Choice

Assignment 2 compares two classification approaches for the DFCA product data. Both methods predict the class attribute Volume. ID3 uses the ID3 training and test files because those files are already discretized. Naive Bayes uses the Bayes training and test files because Weight and Material remain continuous there. That is important because ID3 builds a decision tree from discrete splits, while Naive Bayes calculates probability scores and can handle continuous values using a probability density formula.

Assignment 2 Method Choice Map

Method	What it produces	Data used	What I would say out loud
ID3	A decision tree and readable IF-THEN rules.	ID3 training and test files.	ID3 is easier to explain because it produces a tree and rules.
Naive Bayes	Probability scores for each possible Volume class.	Bayes training and test files.	Naive Bayes is probability-based, so it shows which class has the strongest evidence.

3.1 ID3 Pre-Pruning and Split Selection

For ID3, the first major idea is that the program chooses the attribute that creates the cleanest split. It starts by calculating MC, which is entropy. Entropy measures how mixed the Volume classes are. Then the program calculates WMC for each candidate attribute. WMC measures how mixed the data would be after splitting on that attribute. The gain is the improvement from the split. ID3 chooses the attribute with the highest gain. In my output, weight had the highest gain at the root, so weight became the root split.

Pre-pruning happens while the tree is being built. The program calculates alpha for each branch. Alpha is the dominant class count divided by the number of records in that branch. Then the program compares alpha to T1. Since T1 was 0.80, and both branches had alpha values below 0.80, the branches were stopped early and assigned their dominant Volume classes.

Assignment 2, 3.1 Formula Map, ID3 Pre-Pruning and Split Selection

Process	Formula	Actual calculation from output table	What each variable means	What I would say out loud
Starting MC or entropy	$MC(D) = -\sum(p_i \log_2(p_i))$	$MC(D) = -((78/365)\log_2(78/365) + (56/365)\log_2(56/365) + (35/365)\log_2(35/365) + (44/365)\log_2(44/365) + (28/365)\log_2(28/365) + (124/365)\log_2(124/365)) = 2.396302$	D = current dataset. p_i = proportion of each Volume class.	This tells me how mixed the Volume classes are before any split.
Weighted MC after split	$WMC(A) = \sum((n_v / n) * MC(D_v))$	$WMC(\text{weight}) = (179/365)(1.955150) + (186/365)(2.326753) = 2.144515$	A = candidate attribute. D_v = branch after splitting. n_v = branch size. n = total records.	This tells me how mixed the branches would be after splitting by weight.
Information gain	$\text{Gain}(A) = MC(D) - WMC(A)$	$\text{Gain}(\text{weight}) = 2.396302 - 2.144515 = 0.251787$	$\text{Gain}(A)$ = how much the split improves class separation.	The larger the gain, the better the split.
Best split	Best split = attribute with maximum $\text{Gain}(A)$	weight had the highest root gain, 0.251787, so ID3 selected weight as the root split.	maximum means largest gain among candidate attributes.	ID3 picked weight because it reduced class mixture the most.
Mixture ratio alpha	$\alpha = c1 / F1 $	For weight = 1, $\alpha = 99 / 179 = 0.5531$. For weight = 2, $\alpha = 74 / 186 = 0.3978$. c1 = dominant class count. F1	= total records in the branch.	Alpha tells me how dominant the strongest Volume class is inside that branch.
T1 pre-pruning decision	If $\alpha < T1$, stop splitting and assign the dominant class.	$T1 = 0.80$. Since $0.5531 < 0.80$, weight = 1 was pre-pruned. Since $0.3978 < 0.80$, weight = 2 was also pre-pruned.	$T1$ = user-selected pre-pruning threshold.	A higher $T1$ stops branches sooner. In this run, both root branches stopped early.

3.2 ID3 Post-Pruning

Post-pruning happens after the tree has already been built. The program checks whether removing a subtree would cause only a small enough loss in correct classifications. The formula compares the loss from pruning to the allowed pruning tolerance. In my output, the root subtree was not removed because the loss was too large.

Assignment 2, 3.2 Formula Map, ID3 Post-Pruning

Process	Formula	Actual calculation from output table	What each variable means	What I would say out loud
Post-pruning test	$(N - M) / Q \leq gK$	$(21 - 16) / 44 \leq 0.010 * 2$	N = number correct before pruning. M = number correct after pruning. Q = total test records. g = user-selected pruning parameter. K = number of branches.	This checks whether pruning loses only an acceptable amount of accuracy.
Left side	$(N - M) / Q$	$(21 - 16) / 44 = 5 / 44 = 0.1136$	N - M = lost correct predictions. Q = 44 test records.	The left side is the accuracy loss caused by pruning.
Right side	gK	$0.010 * 2 = 0.0200$	$g = 0.010$. K = 2 branches.	The right side is the allowed loss.
Final pruning decision	Remove subtree only if left side $\leq$ right side.	$0.1136 \leq 0.0200$ is false, so the subtree was kept.	false means the pruning condition was not satisfied.	The tree was kept because removing the subtree would lose too many correct predictions.

3.3 ID3 Results

For ID3, the final accuracy after post-pruning was 47.73%. There were 44 test records, and the model correctly classified 21 of them. In a multi-class confusion matrix, the correct predictions are on the diagonal, where the actual Volume class matches the predicted Volume class. Precision and recall can also be discussed by class. For example, for Volume 6, ID3 had 13 true positives, 11 false positives, and 3 false negatives.

Assignment 2, 3.3 Formula Map, ID3 Results

Process	Formula	Actual calculation from output table	What each variable means	What I would say out loud
Accuracy	accuracy = correct predictions / total test records	ID3 accuracy = 21 / 44 = 0.4773 = 47.73%	correct predictions = diagonal cells in the confusion matrix.	Accuracy tells how often ID3 was right overall.
True positive for Volume 6	TP_6 = actual 6 and predicted 6	TP = 13	TP = true positives for Volume 6.	ID3 correctly predicted 13 actual Volume 6 records as Volume 6.
False negative for Volume 6	FN_6 = actual 6 but predicted not 6	FN = 3	FN = actual Volume 6 records missed by the model.	ID3 missed 3 records that were really Volume 6.
False positive for Volume 6	FP_6 = predicted 6 but actual not 6	FP = 11	FP = records incorrectly assigned to Volume 6.	ID3 over-predicted Volume 6 for 11 records from other classes.
Precision for Volume 6	precision = TP / (TP + FP)	13 / (13 + 11) = 13 / 24 = 0.54	TP = correct Volume 6 predictions. FP = wrong Volume 6 predictions.	Precision asks, when ID3 predicted Volume 6, how often was it correct?
Recall for Volume 6	recall = TP / (TP + FN)	13 / (13 + 3) = 13 / 16 = 0.81	FN = actual Volume 6 records the model missed.	Recall asks, out of all real Volume 6 records, how many did ID3 find?

4.1 Bayes Class Priors

Naive Bayes starts with the class priors. A prior is the starting probability of a class before looking at the product attributes. Volume 6 has the largest prior because it has the largest number of training records. It has 124 out of 365 training records, so its prior probability is 0.3397. That matters because every final Bayes score begins with the prior. A class with a larger prior starts stronger, but it still has to be supported by the attribute evidence.

Assignment 2, 4.1 Formula Map, Bayes Class Priors

Process	Formula	Actual calculation from output table	What each variable means	What I would say out loud
Class prior	$P(C) = \text{count}(C) / \text{total training records}$	$P(\text{Volume} = 6) = 124 / 365 = 0.3397$	C = one Volume class. count(C) = training records in that class.	This is the starting chance before looking at the product details.
Largest prior	largest P(C) comes from largest class count	Volume 6 has 124 training records, which is more than any other Volume class.	Training count = examples available for that class.	Volume 6 starts with the strongest base probability, but the attribute evidence can still change the final prediction.

4.2 Smoothing and Continuous Attributes

For categorical attributes, Naive Bayes calculates the probability of each attribute value inside each Volume class. This is written as $P(A = v | C)$, which means the probability that attribute A has value v, given class C. Smoothing was needed because some attribute values had zero counts for some Volume classes. Without smoothing, one zero probability would make the entire class score zero because the probabilities are multiplied together. With Laplace smoothing, a missing count becomes a small probability instead of an automatic zero.

For Weight and Material, the program treated the values as continuous. Instead of exact value counts, it used the mean and sample standard deviation for each Volume class, then applied the Gaussian probability density formula.

Assignment 2, 4.2 Formula Map, Smoothing and Continuous Attributes

Process	Formula	Actual calculation from output table	What each variable means	What I would say out loud
Conditional probability	$P(A = v C) = \text{count}(A = v \text{ and } C) / \text{count}(C)$	Example idea: $P(\text{Venue} = 1 \text{Volume} = 6)$ uses records where Volume = 6 and Venue = 1.	A = attribute. v = attribute value. C = Volume class.	This asks how common a test-record value is inside one class.
Laplace smoothing	$P(A = v C) = (\text{count} + 1) / (\text{class_count} + \text{domain_size})$	$P(\text{Venue} = 1 \text{Volume} = 6) = (98 + 1) / (124 + 2) = 99 / 126 = 0.7857$	count = matching records. class_count = 124. domain_size = 2 possible Venue values.	Smoothing keeps the probability from becoming too harsh when a combination is rare or missing.
Zero-count problem	If one probability is 0, Score(C) becomes 0 because everything is multiplied.	Without smoothing: $P(\text{color} = 1 \text{Volume} = 1) = 0 / 78 = 0$. With smoothing: $(0 + 1) / (78 + 6) = 1 / 84 = 0.0119$.	6 = number of possible color values.	A zero count should make the class less likely, not automatically impossible.
Continuous mean	mean = sum(values) / n	For Volume 6 Weight, mean = 1.752016 using n = 124 records.	n = number of training records in that Volume class.	The mean is the typical Weight value for that class.

Sample standard deviation	$\text{std} = \sqrt{\text{sum}((x - \text{mean})^2) / (n - 1)}$	For Volume 6 Weight, sample STD = 0.503540.	x = each Weight value. mean = class mean. $n - 1$ = sample correction.	Standard deviation tells how spread out the Weight values are in that class.
Gaussian density	$f(x C) = [1 / (\sqrt{2\pi}) * \sigma] * e^{-(x - \mu)^2 / (2 * \sigma^2)}$	$f(\text{Weight} = 1.50 \text{Volume} = 6) = 0.699009$ using $\mu = 1.752016$ and $\sigma = 0.503540$.	x = test value. μ = mean. σ = standard deviation.	This lets Bayes score continuous values without requiring exact matches.

4.3 Naive Bayes Results

Naive Bayes calculates one score for each possible Volume class. The score starts with the prior, then multiplies the categorical probability product and the continuous probability product. For test row 1, the Volume 6 score was the largest, so the model predicted Volume 6. Overall, Naive Bayes correctly classified 20 out of 44 test records, which gives an accuracy of 45.45%.

Assignment 2, 4.3 Formula Map, Naive Bayes Results

Process	Formula	Actual calculation from output table	What each variable means	What I would say out loud
Class score	$\text{Score}(C) = P(C) * \text{categorical_product} * \text{continuous_product}$	For test row 1, Volume 6 score = $0.3397260274 * 0.0001116448 * 0.1375198007 = 0.00000521594$	$P(C)$ = prior. categorical_product = multiplied categorical probabilities. continuous_product = Weight and Material density product.	Bayes builds one score for every possible Volume class.
Prediction rule	predicted class = class with max $\text{Score}(C)$	For test row 1, Volume 6 score = 0.00000521594, larger than Volume 4 score = 0.000000721005 and Volume 2 score = 0.000000667963.	max means choose the largest class score.	The biggest score wins, so test row 1 was predicted as Volume 6.
Accuracy	accuracy = correct predictions / total test records	Naive Bayes accuracy = $20 / 44 = 0.4545 = 45.45\%$	correct predictions = diagonal of confusion matrix.	Accuracy tells how often Bayes was right overall.
True positive for Volume 6	TP_6 = actual 6 and predicted 6	TP = 12	TP = correct Volume 6 predictions.	Bayes correctly predicted 12 actual Volume 6 records.
False negative for Volume 6	FN_6 = actual 6 but predicted not 6	FN = 4	FN = actual Volume 6 records missed by Bayes.	Bayes missed 4 actual Volume 6 records.
False positive for Volume 6	FP_6 = predicted 6 but actual not 6	FP = 9	FP = records incorrectly assigned to Volume 6.	Bayes over-predicted Volume 6 for 9 records.
Precision for Volume 6	precision = $\text{TP} / (\text{TP} + \text{FP})$	$12 / (12 + 9) = 12 / 21 = 0.57$	TP = true positives. FP = false positives.	Precision asks, when Bayes predicted Volume 6, how often was it right?
Recall for Volume 6	recall = $\text{TP} / (\text{TP} + \text{FN})$	$12 / (12 + 4) = 12 / 16 = 0.75$	FN = false negatives.	Recall asks, out of all real Volume 6 records, how many did Bayes find?

Assignment 2 Closing Comparison

In this run, ID3 had an accuracy of 47.73%, and Naive Bayes had an accuracy of 45.45%. ID3 performed slightly better in overall accuracy. The bigger difference is interpretability. ID3 gives a decision tree and IF-THEN rules, so it is easier to explain. Naive Bayes gives probability scores, so it is useful when the company wants to see which class had the strongest probability evidence. Since the company is deciding between the two methods, I would say ID3 is slightly better for this run because it had the higher accuracy and the result is easier to explain to nontechnical users. But Naive Bayes is still useful because it handled Weight and Material as continuous attributes and showed probability-based evidence for each prediction.

Quick Professor Question Table

Question	Short conversational answer
Why do Assignment 1 and Assignment 2 look different?	Assignment 1 is a notebook-style pipeline because that version was easiest to present from. Assignment 2 is a VS Code Python pipeline because the assignment requires the user to choose ID3 or Bayes and run the selected program.
Why did Assignment 1 accuracy look weak?	The surviving rules mostly predicted treat = 1, so the model found all treat = 1 records but mislabeled the treat = 0 records.
Why use no default rule for Assignment 1?	The original Assignment 1 directions did not specify a fallback rule, so the official result uses only the matched Format-1 rules.
Why does ID3 use entropy and gain?	Entropy measures class mixture. Gain measures how much cleaner a split makes the data. ID3 chooses the attribute with the

	highest gain.
Why did ID3 pre-prune both branches?	With $T1 = 0.80$, both alpha values were below the threshold, so the branches stopped early and predicted the dominant class.
Why was the ID3 subtree not post-pruned?	The loss from removing it was 0.1136, but the allowed loss was only 0.0200, so pruning would have hurt too much.
Why did Bayes need smoothing?	Some categorical value and class combinations had zero counts. Since Bayes multiplies probabilities, one zero would make the whole class score zero.
Why are Weight and Material handled differently in Bayes?	In the Bayes files, Weight and Material are continuous, so the program uses mean, standard deviation, and the Gaussian density formula instead of simple categorical counts.
Which Assignment 2 method would you recommend?	Based on this run, ID3 had slightly better accuracy and is easier to explain, so I would recommend ID3 for this dataset.

Final Closing Script

That completes both assignments. In Assignment 1, the focus was preprocessing, discretization, association rule mining, and prediction from rules. In Assignment 2, the focus was comparing ID3 and Naive Bayes as classification methods. I kept the calculations visible through the reports and Excel outputs so the results can be traced back to the formulas. If you want to inspect any specific calculation, I can open the matching output file and walk through the exact row.